47-SVPWM 18

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

上节课我们讲到电位器给到一个新的位置，给定到 SVPWM 模块的值呢，不会立即变化，而是按照一个增量去变化

```
/*==========================================================================*/
static void PwmDutyControl(void)
{
    unsigned int Temp;

    Temp = tBldc.PwmCount;

    if(Temp == 0)
    {
        tBldc.PwmCount = 10;

        if(tBldc.PotValue > tBldc.DutyInner)
        {
            Temp = tBldc.PotValue - tBldc.DutyInner;
            if(Temp > 100)
                tBldc.DutyInner += 100;
            else if(Temp > 10)
                tBldc.DutyInner += 10;
            else
                tBldc.DutyInner = tBldc.PotValue;
        }
        else
        {
            Temp = tBldc.DutyInner - tBldc.PotValue;
            if(Temp > 100)
                tBldc.DutyInner -= 100;
            else if(Temp > 10)
                tBldc.DutyInner -= 10;
            else
                tBldc.DutyInner = tBldc.PotValue;
        }
    }
}
```

这里就写了这么一个函数来实现这个功能。

```
/*==========================================================================*/
static void PwmDutyControl(void)
{
    unsigned int Temp;

    Temp = tBldc.PwmCount;

    if(Temp == 0)
    {
        tBldc.PwmCount = 10;

        if(tBldc.PotValue > tBldc.DutyInner)
        {
            Temp = tBldc.PotValue - tBldc.DutyInner;
            if(Temp > 100)
```

这个 Pwmcount 是一个计数变量，用这个计数变量来设定时间。这个值呢，如果不是 0 的话，会 1ms 递减一次。所以，这里设定初值为 10 的话，就是 10ms。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
static void PwmDutyControl(void)
{
    unsigned int Temp;

    Temp = tBldc.PwmCount;

    if(Temp == 0)
    {
        tBldc.PwmCount = 10;

        if(tBldc.PotValue > tBldc.DutyInner)
        {
            Temp = tBldc.PotValue - tBldc.DutyInner;
            if(Temp > 100)
                tBldc.DutyInner += 100;
            else if(Temp > 10)
                tBldc.DutyInner += 10;
            else
                tBldc.DutyInner = tBldc.PotValue;
        }
```

所以，这段代码也就是 10ms 执行一次。然后我们再来分析看这段代码实现了什么功能？

```
if(Temp == 0)
{
    tBldc.PwmCount = 10;

    if(tBldc.PotValue > tBldc.DutyInner)
    {
        Temp = tBldc.PotValue - tBldc.DutyInner;
        if(Temp > 100)
            tBldc.DutyInner += 100;
        else if(Temp > 10)
            tBldc.DutyInner += 10;
        else
            tBldc.DutyInner = tBldc.PotValue;
    }
```

这里呢，判断了一下处理之后的旋钮瞬时值　跟赋值给电压矢量圆大小的值的大小关系。如果此时是旋钮瞬时值大的话，也就说明我们拧动的方向是值变大的方向。此时呢，求取了它们的差值，而因为是旋钮的瞬时值大，所以差值是正的。然后呢，我们可以根据差值的大小来决定递增值的大小。

如果差值很大，且大于 100，则就按照 100 的增量递增到给定值。如果差值在 10~100 之间，就按照 10 的增量递增到给定值。如果差值小于 10，就直接给。

```
if(Temp == 0)
{
    tBldc.PwmCount = 10;

    if(tBldc.PotValue > tBldc.DutyInner)
    {
        Temp = tBldc.PotValue - tBldc.DutyInner;
        if(Temp > 100)
            tBldc.DutyInner += 100;
        else if(Temp > 10)
            tBldc.DutyInner += 10;
        else
            tBldc.DutyInner = tBldc.PotValue;
    }
    else
    {
        Temp = tBldc.DutyInner - tBldc.PotValue;
        if(Temp > 100)
            tBldc.DutyInner -= 100;
        else if(Temp > 10)
            tBldc.DutyInner -= 10;
        else
            tBldc.DutyInner = tBldc.PotValue;
    }
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

那么，同理呢，这里就是旋钮往小的方向调整的时候。思路是一样的，也是求取差值，只是把加变为了减。

```
/*==========================================================================*/
static void PwmDutyControl(void)
{
    unsigned int Temp;

    Temp = tBldc.PwmCount;

    if(Temp == 0)
    {
        tBldc.PwmCount = 10;

        if(tBldc.PotValue > tBldc.DutyInner)
        {
            Temp = tBldc.PotValue - tBldc.DutyInner;
            if(Temp > 100)
                tBldc.DutyInner += 100;
            else if(Temp > 10)
                tBldc.DutyInner += 10;
            else
                tBldc.DutyInner = tBldc.PotValue;
        }
        else
        {
            Temp = tBldc.DutyInner - tBldc.PotValue;
            if(Temp > 100)
                tBldc.DutyInner -= 100;
            else if(Temp > 10)
                tBldc.DutyInner -= 10;
            else
                tBldc.DutyInner = tBldc.PotValue;
        }
    }
}
```

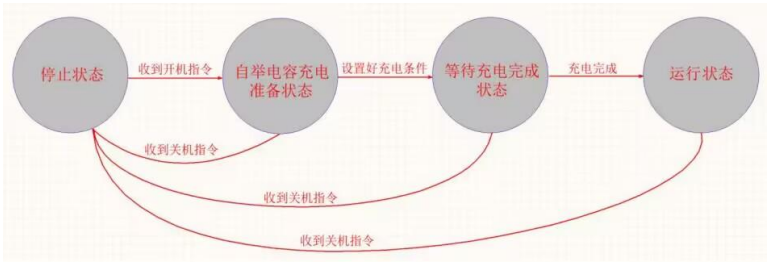
所以，这样处理之后，这个值的变化就是有一个斜坡的。

```
else if(tBldc.Staus == 3) //3:Run
{
    IOCON1 = IOCON_HPWM_LPWM;
    IOCON2 = IOCON_HPWM_LPWM;
    IOCON3 = IOCON_HPWM_LPWM;

    GetRotorInfo();
    if(tBldc.Direction)
        tCore.qEAngle = tBldc.Eangle_Rotor + EANGLE90;
    else
        tCore.qEAngle = tBldc.Eangle_Rotor - EANGLE90;
    tUm.Percent = tBldc.DutyInner;
    UsOut();
}
```

这个值呢，就是我们上次课讲过的赋值给 Percent 的值，也就是作为矢量圆半径的大小的设定值。这样的话，就实现了我们拧电位器，可以等效人为调整矢量圆的半径大小。

上面的控制没有涉及到电机的具体转速。我们继续啊。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

前面的课呢，我们也讲到过这幅状态图，我们说，如果没有开机指令，则系统状态就会一直保持在停止状态。当收到开机指令时，就会跳到自举电容充电准备状态，然后最终到运行状态，让电机运转。这个开机指令呢，我们这里也可以用旋钮的值实现。我们也可以判断，旋钮的值大于 0，就开机，等于 0 就关机。

```
/*==========================================================================*/
static void BldcRunStopControl(void)
{
    if(Motor_Flag.Bits.Status == 0)
    {
        if(tBldc.PotValue > 0)
        {
            Motor_Flag.Bits.Status = 1;
            BldcStartUp();
            tBldc.PwmCount = 10;
        }
    }
    else if(tBldc.PotValue == 0)
    {
        Motor_Flag.Bits.Status = 0;
        BldcStop();
    }
}
```

所以呢，这里可以写这么一个函数。

```
/*==========================================================================*/
typedef union
{
    unsigned int Word;
    struct
    {
        unsigned Status:1;
        unsigned ReqOff:1;
        unsigned Unused:14;
    }Bits;
}UNION_MOTORFLAGS;
UNION_MOTORFLAGS Motor_Flag;
```

这个地方呢，定义了一个联合体类型。用于实现一些位变量的访问。Status 这个位呢，表示整个系统的运行状态。

```
/*==========================================================================*/
static void BldcRunStopControl(void)
{
    if(Motor_Flag.Bits.Status == 0)
    {
        if(tBldc.PotValue > 0)
        {
            Motor_Flag.Bits.Status = 1;
            BldcStartUp();
            tBldc.PwmCount = 10;
        }
    }
    else if(tBldc.PotValue == 0)
    {
        Motor_Flag.Bits.Status = 0;
        BldcStop();
    }
}
/*==========================================================================*/
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

如果说系统处于停止状态，则这个 Status 就是 0。此时呢，我们在这个状态去检测有没有开机指令。

```
if(Motor_Flag.Bits.Status == 0)
{
    if(tBldc.PotValue > 0)
    {
        Motor_Flag.Bits.Status = 1;
        BldcStartUp();
        tBldc.PwmCount = 10;
    }
}
else if(tBldc.PotValue == 0)
```

检测的方法呢，就是判断我们处理之后的旋钮采样瞬时值是否大于 0，如果大于 0，就可以作为开机指令，让状态机发生跳转。

```
static void BldcRunStopControl(void)
{
    if(Motor_Flag.Bits.Status == 0)
    {
        if(tBldc.PotValue > 0)
        {
            Motor_Flag.Bits.Status = 1;
            BldcStartUp();
            tBldc.PwmCount = 10;
        }
    }
```

满足开机条件的话，会把 Status 置为 1，说明系统开始运行了，同时呢这个位也作为开机后，判断关机条件使用。

```
/*=============================================================================
static void BldcStartUp(void)
{
    tBldc.Staus = 1;
    PDC1 = MINDUTY;
    PDC2 = MINDUTY;
    PDC3 = MINDUTY;
}
```

同时呢，　也调用了这么一个函数。函数进来呢，就对状态机的状态进行修改了。让这个状态变为 1 了，也就跳转到自举电容充电的准备状态。

```
/*=============================================================================*/
static void BldcStartUp(void)
{
    tBldc.Staus = 1;
    PDC1 = MINDUTY;
    PDC2 = MINDUTY;
    PDC3 = MINDUTY;
}
```

然后，也对比较值寄存器赋值了一个最小值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
#define    MAXDUTY    2760
#define    MINDUTY    300
```

实际上设定这个值呢，也对自举电容充电的下管占空比有影响。值越小呢，下管占空比就大。值越大呢，下管占空比就小.

```
static void BldcRunStopControl(void)
{
    if(Motor_Flag.Bits.Status == 0)
    {
        if(tBldc.PotValue > 0)
        {
            Motor_Flag.Bits.Status = 1;
            BldcStartUp();
            tBldc.PwmCount = 10;
        }
    }
    else if(tBldc.PotValue == 0)
    {
        Motor_Flag.Bits.Status = 0;
        BldcStop();
    }
}
```

然后呢，这个变量赋值 10，也是设定电压矢量圆给定值变量调整的速度，给一个初值。这就是开机指令的处理，大家看下，可以理解吧？

```
static void BldcRunStopControl(void)
{
    if(Motor_Flag.Bits.Status == 0)
    {
        if(tBldc.PotValue > 0)
        {
            Motor_Flag.Bits.Status = 1;
            BldcStartUp();
            tBldc.PwmCount = 10;
        }
    }
    else if(tBldc.PotValue == 0)
    {
        Motor_Flag.Bits.Status = 0;
        BldcStop();
    }
}
```

接着，如果系统在运行状态，并且旋钮值是 0 的时候，也就会满足关机条件。满足关机条件的话，就会把系统运行状态标志置为 0，也为后面再次开机做准备。同时呢，需要把状态机的状态设置为停止状态。

```
static void BldcStop(void)
{
    tBldc.Staus = 0;
    IOCON1 = IOCON_HL_INVALID;
    IOCON2 = IOCON_HL_INVALID;
    IOCON3 = IOCON_HL_INVALID;
    tBldc.DutyInner = 655;
    PDC1 = 0;
    PDC2 = 0;
    PDC3 = 0;
}
```

进入到这个函数，把状态变量设置为 0 了，就可以让状态机跳到停止状态。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=====================================================================*/
static void BldcStop(void)
{
    tBldc.Staus = 0;
    IOCON1 = IOCON_HL_INVALID;
    IOCON2 = IOCON_HL_INVALID;
    IOCON3 = IOCON_HL_INVALID;
    tBldc.DutyInner = 655;
    PDC1 = 0;
    PDC2 = 0;
    PDC3 = 0;
}
```

同时呢，也强制关闭 6 路 PWM 输出，并且设定比较值寄存器为 0。然后设定电压矢量圆半径为 1%，这样确保系统停止，没有 PWM 输出，也为后面再次启动做准备。

```
/*=====================================================================*/
static void BldcRunStopControl(void)
{
    if(Motor_Flag.Bits.Status == 0)
    {
        if(tBldc.PotValue > 0)
        {
            Motor_Flag.Bits.Status = 1;
            BldcStartUp();
            tBldc.PwmCount = 10;
        }
    }
    else if(tBldc.PotValue == 0)
    {
        Motor_Flag.Bits.Status = 0;
        BldcStop();
    }
}
```

所以，这个函数调用后，就可以实现，开机，关机的控制。

```
/*=====================================================================*/
void BldcApp(void)
{
    AdcModule();
    BldcRunStopControl();
    PwmDutyControl();
}
/*=====================================================================*/
```

那么，我们刚才分析的这两个函数会被调用，因为它们对实时性没有要求，所以被放到了主函数中去调用。

```
/*=====================================================================*/
void StartProgram(void)
{
    unsigned int Temp;

    ENIE();
    Initialize();
    MainDelayTime = 20000; //1s

    do
    {
        Temp = MainDelayTime;
    }while(Temp != 0);

    while(1)
    {
        BldcApp();
        ProtectApp();
        LedApp(tProtect.Code);
    }
}
```

也就是这个地方。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=====================================================================*/
static void AdcModule(void)
{
    AdcPotProcess();
    tBldc.BusValue = GetVbusVoltage(tAdc.An3.ValueAverage);
}
/*=====================================================================*/
```

然后呢，我们刚才分析的旋钮采样瞬时值的处理函数，是在这调用的。本质呢，也是放到主函数中执行的。所以，到目前为止，大家就可以通过电位器来控制电机启动，停止，加速，减速了，基本就实现了基础的 SVPWM 控制 。下一步呢，我们可以再做一些基础的故障保护。

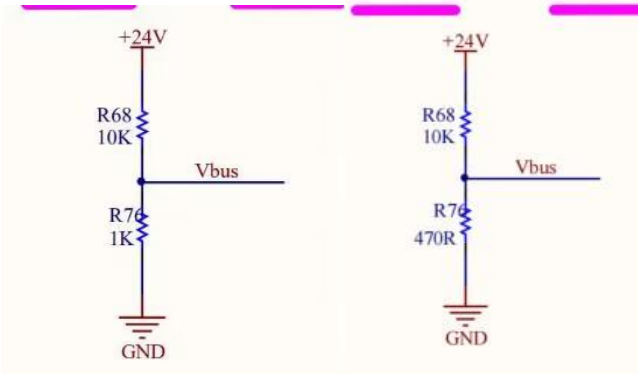
问：能不能做到速度可设置？

答：可以的，用电位器设定转速，再加上速度闭环，用速度环输出来控制 Us 的大小就可以了。

例如，欠压，过压保护，霍尔故障保护。然后，我们 SVPWM 部分的内容就结束了
那么，要做电压保护的话，需要把 Vbus 电压获取到。

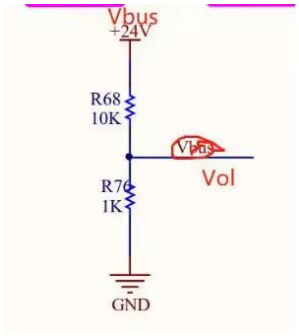
```
/*=====================================================================*/
static void AdcFilter(void)
{
    //Vbus电压 64次平均滤波处理
    tAdc.An3.ValueSum += tAdc.An3.ValueInstant;
    tAdc.An3.ValueNum ++;
    if(tAdc.An3.ValueNum >= 64)
    {
        tAdc.An3.ValueAverage = tAdc.An3.ValueSum >> 6;
        tAdc.An3.ValueSum = 0;
        tAdc.An3.ValueNum = 0;
    }
```

通过均值滤波的方式，可以把 Vbus 电压的平均值给获取到。咱们呢，可以通过这个值，再转换为实际的 Vbus 电压值。那怎么转换呢？大家能不能列一个公式？



这个地方前期更新过，不知道大家手头上的是 1K 还是 470R。这是我们的 Vbus 电压采样电路。我们暂且先按照这个电路讲解，大家需要注意一下自己的学习板硬件电路实际阻值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



为了方便列公式，我们修改一下标号。

$$Vol = \frac{R76}{R68+R76} Vbus$$

Vol 跟 Vbus 的关系是这个。

$$Vol\ (AD) = \frac{Vol}{3.3V} * 1024$$

然后，Vol 的 AD 值对应这个公式。

$$Vol = \frac{R76}{R68+R76} Vbus$$

$$Vol\ (AD) = \frac{Vol}{3.3V} * 1024$$

那么，把它们两者一结合。

$$Vol\ (AD) = \frac{\frac{R76}{R68+R76} Vbus}{3.3V} * 1024$$

就可以得到 Vbus 跟 VolAD 值的关系。而实际上呢，我们 AD 采样之后，AD 值是已知的，所以就是要根据 AD 值推算对应的 Vbus 值。

$$Vbus = \frac{Vol\ (AD) * 3.3V}{1024} * \frac{R68+R76}{R76}$$

所以，对应的推算公式就是这个。R68　R76 的值是已知的。

所以，最终的公式就是这个 $$Vbus = \frac{Vol\ (AD) * 36.3V}{1024}$$ 。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================================*/
static unsigned int GetVbusVoltage(unsigned int Vbus_Adc)
{
    UNION_U32 unTemp32;

    unTemp32.U32 = __builtin_muluu(VBUS_FACTOR, Vbus_Adc);
    unTemp32.U32 >>= 10;
    return (unTemp32.Words.Low);
}
/*==========================================================================*/
```

那么，这个公式转换为代码，就是这个函数。

```
/*==========================================================================*/
#define     VBUS_FACTOR         36
#define     VBUS_LOW_LIMIT      18
#define     VBUS_HIGH_LIMIT     30
/*==========版权归张飞实战电子所有，违者必究！
```

36.3 呢，用宏定义代替了

```
/*==========================================================================*/
static void AdcModule(void)
{
    AdcPotProcess();
    tBldc.BusValue = GetVbusVoltage(tAdc.An3.ValueAverage);
}
/*==========================================================================*/
```

这样调用之后，Vbus 就推算出来了，要做电压保护的话，就可以根据这个值去做判断了。下面我们就来看下对应的保护代码处理方式啊

```
26
27          if(tProtect.CountMs)
28              tProtect.CountMs --;
29      }
30  }
31  /*==========================================================================*/
32  static void VoltageDetect(void)
33  {
34      if(tBldc.BusValue > VBUS_HIGH_LIMIT)
35      {
36          if(tProtect.CountMs == 0)
37              tProtect.Code = 0x03;
38      }
39      else if(tBldc.BusValue > VBUS_LOW_LIMIT)
40      {
41          tProtect.CountMs = 10;
42      }
43      else
44      {
45          if(tProtect.CountMs == 0)
46              tProtect.Code = 0x02;
47      }
48  }
49  /*==========================================================================*/
```

大家看下，这里就是判断我们推算出来的电压是否大于设定的过压阈值，或者小于设定的欠压阈值。如果大于过压阈值的话，就报过压故障，如果小于过压阈值的话，就报欠压故障。当然了，满足了过压，欠压条件后，也需要持续一段时间，最终才设定故障代码。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问： Us 归一化后的 1 会跟着 UbuS 值大小变化吗？
答：会的。

```
void ProtectInt(void)
{
    tProtect.Count ++;
    if(tProtect.Count >= 20)
    {
        tProtect.Count = 0;

        if(tProtect.CountMs)
            tProtect.CountMs --;
    }
}
```

然后呢，这个持续时间的计数变量，是在这个函数中递减的，CountMs 变量是 1ms 递减一次。

问： 这里相当于 Q16 乘 Q10,乘出来为 26 位的值，所以右移 10 位变为 Q16？

```
static unsigned int GetVbusVoltage(unsigned int Vbus_Adc)
{
    UNION_U32 unTemp32;

    unTemp32.U32 = __builtin_muluu(VBUS_FACTOR,Vbus_Adc);
    unTemp32.U32 >>= 10;
    return (unTemp32.Words.Low);
}
```

答：$$Vbus = \frac{Vol（AD）* 36.3V}{1024}$$ 按照这个公式理解，整数乘除法。

所以，我们设定初值是 10 的话，递减到 0 对应的时间就是 10ms。也就是满足过压阈值，或者欠压阈值后，且持续 10ms，就会报过压或者欠压故障。

```
void ProtectInt(void)
{
    tProtect.Count ++;
    if(tProtect.Count >= 20)
    {
        tProtect.Count = 0;

        if(tProtect.CountMs)
            tProtect.CountMs --;
    }
}

static void VoltageDetect(void)
{
    if(tBldc.BusValue > VBUS_HIGH_LIMIT)
    {
        if(tProtect.CountMs == 0)
            tProtect.Code = 0x03;
    }
    else if(tBldc.BusValue < VBUS_LOW_LIMIT)
    {
        tProtect.CountMs = 10;
    }
    else
    {
        if(tProtect.CountMs == 0)
            tProtect.Code = 0x02;
    }
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这就是这个过欠压的监测代码。大家看下，这里有没有疑问？之前我们说过有霍尔传感器的异常故障，异常的时候呢，我们也设置了一个异常标志位。

```
else
{
    if (tBldc.Hall_Update_Count < 65535)                    //65535 x 50us
    {
        tBldc.Hall_Update_Count ++;

        if (tBldc.Direction == 0x01)
            Rotor_Angle.U32 += Rotor_Angle_Inc.U32;
        else
            Rotor_Angle.U32 -= Rotor_Angle_Inc.U32;
    }
    else
    {
        if (Motor_Flag.Bits.Status)
            Error_Flag.Bits.Hall = 1;
        else
            tBldc.Hall_Update_Count = 0;
    }
}
tBldc.Eangle_Rotor = Rotor_Angle.Words.High;
}
/*=============================================================================*/
```

霍尔值长时间不更新，有霍尔传感器的异常故障。

```
if((tBldc.Hall_Value_Buf[0] == Temp.Word) && (tBldc.Hall_Value_Buf[1] == Temp.Word))
{
    tBldc.Hall_Value = Temp.Word;
    if (tBldc.Hall_Value != tBldc.Hall_Value_Old)
    {
        tBldc.Hall_Value_Old = tBldc.Hall_Value;

        if ((tBldc.Hall_Value > 0) && (tBldc.Hall_Value < 7))
        {
            tBldc.Hall_Update = 1;
        }
        else
        {
            Error_Flag.Bits.Hall = 1;
        }
    }
}
}
```

或者呢，霍尔值不在正常范围内，这个时候呢，我们可以报一个霍尔信号异常故障。

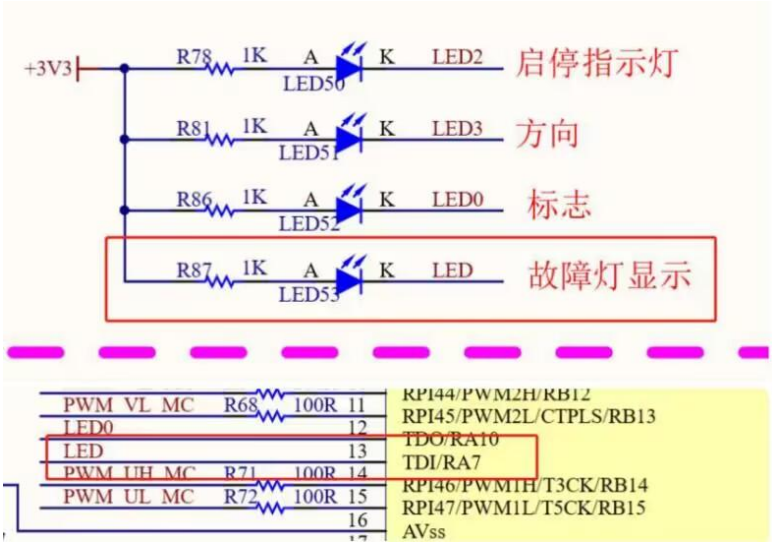
```
/*=============================================================================*/
static void HallDetect(void)
{
    if(Error_Flag.Bits.Hall)
        tProtect.Code = 0x01;
}
/*=================================版权归张飞实战电子所有，违者必究！！！=========
```

所以，这个地方呢，就是判断标志位是 1 的话，就设定了一个故障代码。然后呢，根据这个故障代码，也可以让 LED 灯闪烁，提示异常情况。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```c
#include "..\Include\Protect.h"
/*=============================================================================*/
void ProtectApp(void)
{
    VoltageDetect();
    HallDetect();
    if((tProtect.Code != 0x00) && (tBldc.Staus != 0))
        tBldc.Staus = 0;
}
/*=============================================================================*/
```

然后大家看，这里是故障监测的情况，把刚刚讲的函数，放在这里进行调用。同时呢，如果故障代码不是 0，且系统在运行，就让系统停止，这样的话，达到保护的目的。然后，有故障的话，可以显示故障，根据故障的提示，我们可以看看是什么原因导致系统停机。所以呢，我们这里的话，就使用板上的故障指示 LED 灯来闪烁，提示故障。



根据原理图的连接关系呢，我们可以知道故障指示 LED 灯是连接到 RA7 这个引脚的。并且呢，RA7 是高电平，LED 灯就熄灭，RA7 是低电平，LED 灯就点亮，所以，我们可以来操控下这个 LED 灯啊。

```c
/*=============================================================================*/
void LedApp(unsigned char N)
{
    unsigned int Temp;
    static unsigned char Cnt;

    if(N == 0)
    {
        Cnt = 0;
        tLed.CountMs = LED_OFF_TIME;
        FAULT_LED_PIN = OFF; //LED OFF
    }
    else
    {
        Temp = tLed.CountMs;
```

大家来看下这个函数啊。这个函数呢，带了一个形参 N，这个形参 N 呢，就跟故障代码相对应。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
void StartProgram(void)
{
    unsigned int Temp;

    ENIE();
    Initialize();
    MainDelayTime = 20000; //1s

    do
    {
        Temp = MainDelayTime;
    }while(Temp != 0);

    while(1)
    {
        BldcApp();
        ProtectApp();
        LedApp(tProtect.Code);
    }
}
```

调用的形式就是这样的，把故障代码作为实参传递进来。

```
void LedApp(unsigned char N)
{
    unsigned int Temp;
    static unsigned char Cnt;

    if(N == 0)
    {
        Cnt = 0;
        tLed.CountMs = LED_OFF_TIME;
        FAULT_LED_PIN = OFF; //LED OFF
    }
    else
    {
        Temp = tLed.CountMs;
```

所以呢，我们对 N 进行判断，就可以知道故障的情况。

```
void LedApp(unsigned char N)
{
    unsigned int Temp;
    static unsigned char Cnt;

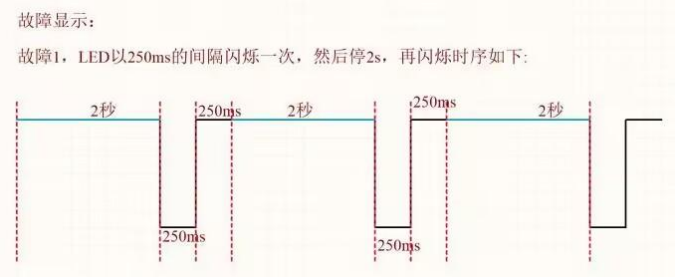
    if(N == 0)
    {
        Cnt = 0;
        tLed.CountMs = LED_OFF_TIME;
        FAULT_LED_PIN = OFF; //LED OFF
    }
    else
    {
        Temp = tLed.CountMs;
```

如果判断出 N 是 0 的话，则说明没有故障。没有故障的话，就让故障 LED 灯保持熄灭。同时呢，也设置计数变量的初值。

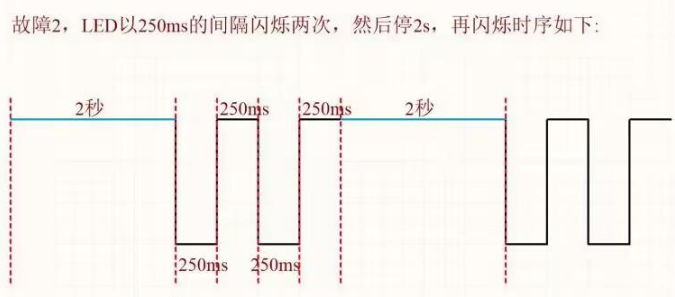
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    }
    else
    {
        Temp = tLed.CountMs;
        if(Temp == 0)
        {
            Cnt ++;
            if(Cnt <= N*2)
            {
                tLed.CountMs = LED_FLICK_TIME;
                FAULT_LED_PIN = ~ FAULT_LED_PIN;
            }
            else
            {
                Cnt = 0;
                tLed.CountMs = LED_OFF_TIME;
                FAULT_LED_PIN = OFF;
            }
        }
    }
}
```

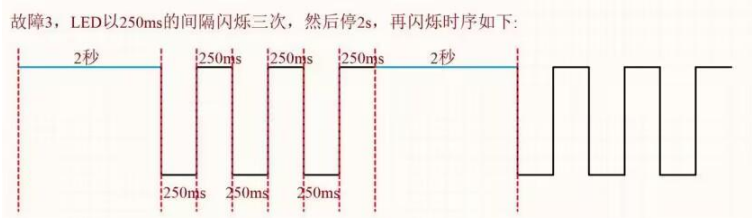
然后呢，到 else 这个分支，就说明有故障了。有故障的话，也就是 N 不等于 0，此时呢，我们需要根据 N 的值来闪烁 LED 灯。



为了方便描述，我们来看下故障提示的波形。如果说 N=1 的话，我们是让 LED 灯以 250ms 的频率来闪烁，闪烁一次后，停 2s，再接着闪烁。如果 N 一直不变的话，那么就一直这样闪烁下去。



如果 N=2 的话，对应的波形就是这样的，以 250ms 闪烁频率闪烁 2 次，然后停 2s，再接着闪烁。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

同理呢，N=3 的话，对应波形是这样的。所以我们需要结合这样的波形，来编写对应的代码。

```
    else
    {
        Temp = tLed.CountMs;
        if(Temp == 0)
        {
            Cnt ++;
            if(Cnt <= N*2)
            {
                tLed.CountMs = LED_FLICK_TIME;
                FAULT_LED_PIN = ~ FAULT_LED_PIN;
            }
            else
            {
                Cnt = 0;
                tLed.CountMs = LED_OFF_TIME;
                FAULT_LED_PIN = OFF;
            }
        }
    }
}
/*==========版权归张飞实战电子所有，违者必究！！！==========*/
```

我们结合刚才的波形来分析这段代码。

```
    else
    {
        Temp = tLed.CountMs;
        if(Temp == 0)
        {
            Cnt ++;
            if(Cnt <= N*2)
            {
                tLed.CountMs = LED_FLICK_TIME;
                FAULT_LED_PIN = ~ FAULT_LED_PIN;
            }
            else
            {
                Cnt = 0;
                tLed.CountMs = LED_OFF_TIME;
                FAULT_LED_PIN = OFF;
            }
        }
    }
}
```

大家看，CountMs 这个变量，还是在中断中 1ms 递减一次。如果我们设定了一个初值，它递减到 0，也就说明对应的时间到了。下面的两个红框呢，就是对它设定初值。

LED_FLICK_TIME　这个值呢，表示闪烁的时候对应的时间值，也就是 250ms。

LED_OFF_TIME　这个值呢，就是熄灭时候的时间值，也就是 2s。

```
/*==========================================================*/
#define    LED_FLICK_TIME      250
#define    LED_OFF_TIME        2000
#define    FAULT_LED_PIN       LATAbits.LATA7
#define    OFF                 1
/*==========================================================*/
```

它们是用宏定义来表示的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
else
{
    Temp = tLed.CountMs;
    if(Temp == 0)
    {
        Cnt ++;
        if(Cnt <= N*2)
        {
            tLed.CountMs = LED_FLICK_TIME;
            FAULT_LED_PIN = ~ FAULT_LED_PIN;
        }
        else
        {
            Cnt = 0;
            tLed.CountMs = LED_OFF_TIME;
            FAULT_LED_PIN = OFF;
        }
    }
}
}
```

所以，一旦进入到这个 if 分支，也就说明我们设定的时间到了。接着呢，我们要根据 N 来闪烁 LED 灯。

此时我们来看，假设 N=1.　第一次进来的时候，Cnt++之后，就从 0 变为 1 了。然后使用 1 跟　N*2 比较，当 N=1 时，显然 Cnt=1 小于 2，　此时呢，就会让 LED 灯对应的 IO 口翻转一次。而因为没有故障的时候，LED 灯是熄灭的，所以这一次翻转，就会让 LED 灯点亮
同时呢，设定闪烁时间，设定好之后呢，又会继续等待时间到。当再次进来的时候，Cnt=2，此时呢，2 还是小于等于 2，所以，还会再进一次这个 if 分支。也就是，LED 灯对应的 IO 口会再次翻转，从点亮变为熄灭。然后，这个状态又会持续 250ms。

```
else
{
    Temp = tLed.CountMs;
    if(Temp == 0)
    {
        Cnt ++;
        if(Cnt <= N*2)
        {
            tLed.CountMs = LED_FLICK_TIME;
            FAULT_LED_PIN = ~ FAULT_LED_PIN;
        }
        else
        {
            Cnt = 0;
            tLed.CountMs = LED_OFF_TIME;
            FAULT_LED_PIN = OFF;
        }
    }
}
}
```

当再一次进来的时候，Cnt=3　>　2 了，所以就会执行 else 分支的程序。此时呢，清零 Cnt 变量，设定熄灭时间为 2s，设定 IO 口为高电平熄灭 LED 灯。这样就完成了一次闪烁动作，如果 N 一直保持为 1，则上述过程就会不断的重复。当 N 等于其它值的时候，也是这样来分析啊。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================================*/
void LedApp(unsigned char N)
{
    unsigned int Temp;
    static unsigned char Cnt;

    if(N == 0)
    {
        Cnt = 0;
        tLed.CountMs = LED_OFF_TIME;
        FAULT_LED_PIN = OFF; //LED OFF
    }
    else
    {
        Temp = tLed.CountMs;
        if(Temp == 0)
        {
            Cnt ++;
            if(Cnt <= N*2)
            {
                tLed.CountMs = LED_FLICK_TIME;
                FAULT_LED_PIN = ~FAULT_LED_PIN;
            }
            else
            {
                Cnt = 0;
                tLed.CountMs = LED_OFF_TIME;
                FAULT_LED_PIN = OFF;
            }
        }
    }
}
/*==========================版权归张飞实战电子所有，违者必究！！！==========*/
```

所以，这个函数，就实现了故障 LED 灯闪烁的功能。根据 N 值的不同，闪烁次数也就不一样。大家如果有疑惑的话，课后也可以仿真一下。

```
16  /*==========================================================================*/
17  void LedInt(void)
18  {
19      LedCount();
20  }
21  /*==========================================================================*/
22  static void LedCount(void)
23  {
24      tLed.Count ++;
25      if(tLed.Count >= 20)
26      {
27          tLed.Count = 0;
28
29          if(tLed.CountMs)
30              tLed.CountMs --;
31      }
32  }
```

那么，这个计数变量呢，也是 1ms 递减一次。

```
2   /*==========================================================================*/
3   //50us Interrupt
4   void __attribute__((section(".main_section"),interrupt,shadow,no_auto_psv)) _AD1Interrupt(void)
5   {
6       IFS0bits.AD1IF = 0;
7       BldcInt();
8       LedInt();
9       ProtectInt();
0       SpiTransmit();
1       if(MainDelayTime)
2           MainDelayTime --;
3   }
4   /*==========================版权归张飞实战电子所有，违者必究！！！==========*/
```

两个函数呢，都放在 50us 中断中执行，所以计数就是 1ms 递减一次。好，那么这样的话，如果出现霍尔异常，或者电压异常，就会有 LED 灯闪烁来提示了。

那么，到目前为止，我们 SVPWM 部分的示例代码，就给大家分析完了。大家有时间的时候，需要多复习复习，有疑问可以问助教，也可以下节课问我。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

我们下次课开始呢，就开始讲 FOC 部分的内容了。那我们今天就先讲到这里哈，大家有什么问题？

问：

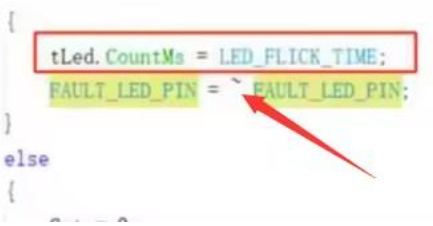
```
/*=============================================================================*/
unsigned int GetVbusVoltage(unsigned int Vbus_Adc)
{
    UNION_U32 unTemp32;

    unTemp32.U32 = __builtin_muluu(VBUS_FACTOR,Vbus_Adc);
    unTemp32.U32 >>= 10;
    return (unTemp32.Words.Low);
}
/*=============================================================================*/
```

这个返回值取低 16 位怎么理解？

答：电压的高 16 位为 0，可以舍弃，这个函数返回值是 16 位，unsigned int 在这个 16 位单片机中表示 16 位。

问：



取反用波浪号吗？

答：位取反，和字节取反是有区别的，单个位取反，用波浪号。

大家早点休息，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。